

Game Variant Fire Brigade

R.A. Diaz Lozada, A.M. Cepeda Villanueva, J.F. Cuervo Huertas

Systems Engineering, School of Engineering
Universidad Distrital Francisco José de Caldas
{radiazl, amcepedav, jfcuerpoh}@udistrital.edu.co
Bogotá, Colombia

Abstract—Forest fire management requires rapid decision-making, balancing fire suppression with civilian evacuation. This paper presents Fire Brigade, a turn-based simulation game on an 8×8 grid that demonstrates greedy and backtracking algorithms. The greedy dispatcher always sends units to the highest-risk cell, determined by a Binary Search Tree (BST). The backtracking evacuator finds safe evacuation routes avoiding burning cells. The game features three difficulty levels, three unit types, four civilian categories, two fire propagation behaviors, a complete scoring system, win/loss conditions, and a JSON-based communication protocol between C++ and Python. This paper presents the complete system design, including architecture, data structures, algorithms, game rules, and communication protocols.

Index Terms—Greedy Algorithm, Backtracking, Binary Search Tree, Linked List, Turn-Based Simulation, JSON Communication.

I. INTRODUCTION

Forest fires represent one of the most challenging natural disasters to manage, requiring rapid decision-making under pressure and efficient allocation of limited resources. Firefighting operations must balance two critical objectives: extinguishing active fires to prevent further spread, and evacuating civilians from danger zones before they become trapped. These challenges make forest fire management an ideal domain for exploring algorithmic solutions, where computational strategies can support or even automate tactical decisions.

Fire Brigade is a turn-based simulation game that models this problem on an 8×8 grid. The player takes command of limited firefighting resources and must decide which units to deploy and which civilians to evacuate, while the algorithms determine the optimal locations and routes. The game is designed to demonstrate the practical application of two classic algorithmic strategies: a greedy algorithm for unit dispatch, which always selects the cell with the highest fire risk; and a backtracking algorithm for civilian evacuation, which exhaustively explores all possible safe paths to the grid boundary.

Previous firefighting simulations, both commercial and academic, have explored resource management and fire spread modeling. Commercial titles such as *Emergency* (Sixteen Tons Entertainment, 2005) and *Fire Department* (Monte Cristo, 2003) emphasize real-time tactical decision-making but do not expose underlying algorithms to the player. Academic simulations like *ForestFire* [2] focus on environmental prediction using cellular automata rather than interactive human decision-making. Fire Brigade addresses this gap by making algorithmic behavior central to gameplay.

The project integrates three distinct technological layers: a C++ engine responsible for core data management and structural logic, a Python module housing the decision-making algorithms, and a Pygame-based graphical interface. Communication between the C++ engine and the Python layer is handled through a JSON bridge, allowing each component to operate independently while efficiently sharing game state.

The C++ engine manages two key data structures: a singly linked list that implements the unit dispatch queue, storing incoming firefighting units (ground crews, trucks, and helicopters) in order of arrival; and a Binary Search Tree (BST) that organizes burning cells by their computed risk score, enabling the greedy algorithm to retrieve the maximum-risk target in $O(\log n)$ time. Risk is calculated based on fire intensity and type, with canopy fires receiving a multiplier of 1.5 due to their more aggressive eight-directional propagation.

The game features three difficulty levels — Easy, Medium, and Hard — each varying the number and type of fires, the composition of civilians on the map, the initial unit allocation, and the rate at which new units arrive. Three built-in variants add strategic depth: a scarce resources variant governing unit types and arrival timing, a civilian priorities variant assigning different point values and movement speeds to four civilian categories (Child, Adult, Elderly, Family Group), and a dynamic fire behavior variant introducing two fire propagation modes with distinct spread patterns.

The development effort was distributed across three complementary roles, each aligned with the architectural layers:

- **Jeison Felipe Cuervo Huertas** — Engine Developer: implemented the C++ core, including the linked list dispatch queue, the BST risk-ranking structure, and the JSON bridge connecting the engine to the Python layer.
- **Andres Mauricio Cepeda Villanueva** — Algorithm Developer: implemented the Greedy dispatcher and the Backtracking evacuator in Python, including tie-breaking logic and path optimization.
- **Raul Andres Diaz Lozada** — UI Developer: built the Pygame interface, rendering the 8×8 grid, fire propagation animations, evacuation sequences, and the score and resource HUD.

II. METHODS AND MATERIALS

A. System Architecture

The system comprises three independent layers:

- **C++ Engine:** Manages game state, fire propagation, unit queue (linked list), and BST for risk ranking.
- **Python Module:** Executes Greedy dispatcher or Backtracking evacuator based on player action.
- **Pygame Interface:** Renders grid, fires, civilians, HUD, and captures player input.

Communication occurs via JSON. The engine serializes state after each turn; Python returns decisions as JSON responses.

B. Data Structures

1) *Linked List – Unit Dispatch Queue:* FIFO queue implemented as singly linked list. Each node stores unit type and arrival turn.

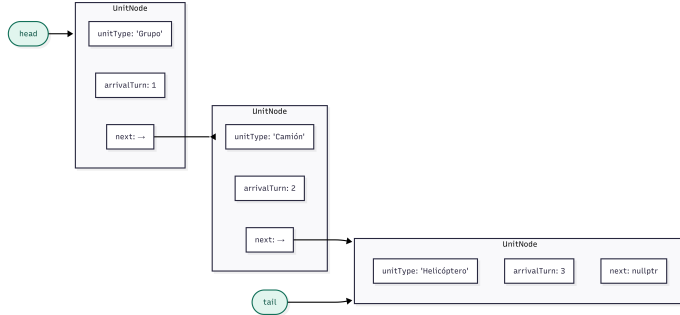


Fig. 1. Linked List Structure

2) *Binary Search Tree – Risk Levels:* BST organizes burning cells by risk value. Each node stores risk level, coordinates, fire type, and child pointers.

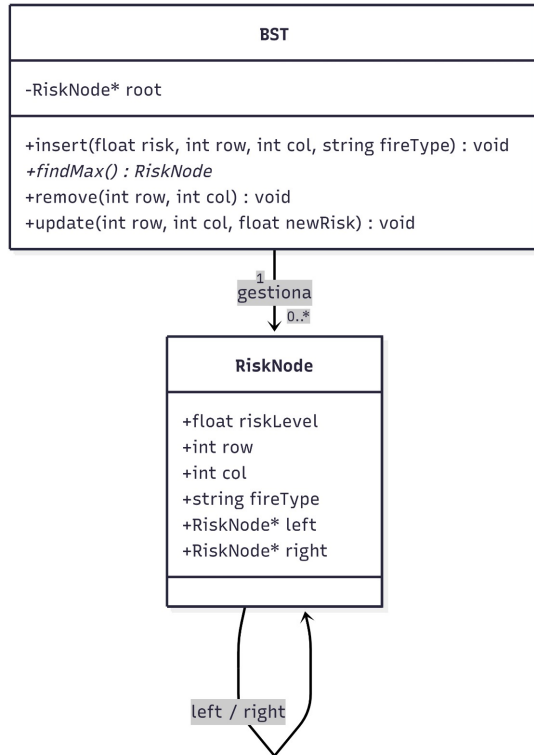


Fig. 2. BST Node Structure

Risk is calculated as: $Risk = intensity \times (1.5 \text{ if canopy, else } 1.0)$.

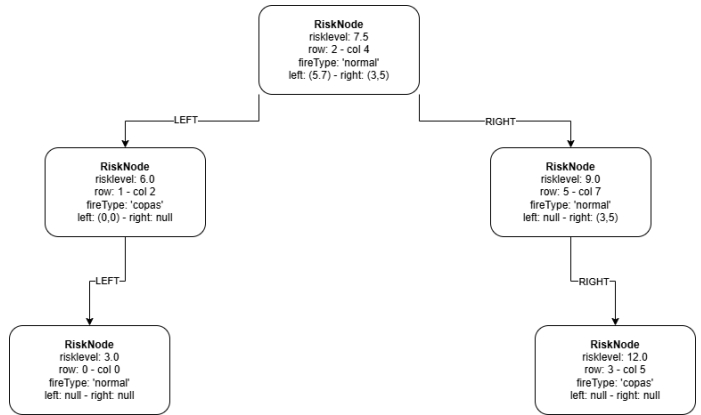


Fig. 3. Example BST Nodes with Tree Structure

C. Algorithms

1) *Greedy Dispatcher:* Selects cell with highest risk from BST. Ties broken by: (1) canopy fire preference, (2) smallest coordinates.

Algorithm 1 Greedy Dispatcher

Require: BST ordered by risk

Ensure: target cell (row, column)

- 1: target $\leftarrow$ bst.getHighestRiskCell()
- 2: **if** multiple cells have same risk **then**
- 3: **if** canopy fire exists **then**
- 4: target $\leftarrow$ select canopy fire
- 5: **else**
- 6: target $\leftarrow$ select smallest row, then column
- 7: **end if**
- 8: **end if**
- 9: **return** target

2) *Backtracking Evacuator:* Finds shortest safe route to border using depth-first search.

Algorithm 2 Backtracking Evacuation

Require: start position, obstacles matrix

Ensure: route, success

- 1: visited $\leftarrow$ False matrix
- 2: explore(start, visited, [])
- 3: **if** bestRoute found **then**
- 4: **return** bestRoute, True
- 5: **else**
- 6: **return** [], False
- 7: **end if**

Algorithm 3 Explore (Recursive)

```

1: Append position to currentRoute
2: if position on border then
3:   update bestRoute; RETURN True
4: end if
5: visited[pos] ← True
6: for each movement in [(-1,0),(1,0),(0,-1),(0,1)] do
7:   if valid move then
8:     explore(newPos)
9:   end if
10: end for
11: visited[pos] ← False; remove last from route

```

D. Game ComponentsTABLE I
UNIT TYPES

Unit	Effect	Arrival
Crew	Extinguishes 1 cell + reduces neighbor risk by half	2-4
Truck	Extinguishes 1 cell + reduces neighbor risk by half	4-6
Helicopter	Extinguishes 1 cell + protects neighbors for 2 turns	6-8

TABLE II
CIVILIAN VALUES

Type	Points
Child	100
Adult	50
Elderly	75
Family Group	150

TABLE III
FIRE TYPES

Type	Spread	Risk Mult.
Normal	4 directions	1.0
Canopy	8 directions	1.5

Propagation Rules:

- Fire propagates every **2 turns**, not every turn.
- Newly ignited cells inherit **80% of the source cell's intensity**.
- Protected cells (affected by a Helicopter) do not ignite during propagation.

E. Game Rules

1) *Scoring System*: Points are awarded according to the following rules:

- Extinguishing a normal fire cell: $+10 \times \text{risk_level}$
- Extinguishing a canopy fire cell: $+15 \times \text{risk_level}$
- Evacuating a child: +100 points
- Evacuating an adult: +50 points
- Evacuating an elderly civilian: +75 points
- Evacuating a family group: +150 points
- Civilian burned: -100 points (civilian is removed from the map)
- All fires extinguished bonus: +200 points
- All civilians evacuated bonus: +300 points

2) *Win Conditions*: The player wins when both of the following conditions are satisfied:

- All fires on the grid have been extinguished.
- All civilians present at the start of the level have been successfully evacuated.

3) *Loss Conditions*: The player loses if either condition occurs:

- More than half of the initial civilians are burned (e.g., 4 initial civilians → third burn triggers defeat).
- Fire spreads to cover more than 80% of the grid (more than 51 out of 64 cells).

4) *Turn Structure*: Each turn follows this sequence:

- 1) **Unit arrival**: New units arrive according to difficulty schedule and are appended to the linked list queue.
- 2) **Player actions**: The player may deploy units (Greedy chooses target) or evacuate civilians (Backtracking finds route). When a Crew or Truck deploys, the risk of neighboring cells is reduced by half. Helicopter protects neighbors for 2 turns.
- 3) **Fire propagation (every 2 turns)**: Active fires spread to neighbors (4 directions for normal fire, 8 for canopy). Newly ignited cells inherit 80% of the source cell's intensity. Protected cells do not ignite.
- 4) **State update**: BST is updated, win/loss conditions are checked, and state is serialized to JSON.

F. Difficulty LevelsTABLE IV
DIFFICULTY PARAMETERS

Parameter	Description
Normal Fires	# of standard fires (4-direction spread)
Canopy Fires	# of aggressive fires (8-direction spread, $\times 1.5$ risk)
Civilians	Types of civilians present on the map
Initial Units	Firefighting units available at start
Crew Arrival	Turns until a new Crew unit arrives
Truck Arrival	Turns until a new Truck unit arrives
Helicopter Arrival	Turns until a new Helicopter arrives

TABLE V
DIFFICULTY LEVEL VALUES

Easy	Medium	Hard
2	2	2
0	1	3
Adult, Child	Adult, Child, Elderly	Adult, Child, Elderly, Family
Crew, Truck, Heli	Crew, Truck, Heli	Crew, Truck
2	3	4
4	5	6
6	7	8

G. JSON Communication Protocol

Communication between C++ and Python occurs exclusively through JSON files. This stateless interface keeps the layers decoupled.

1) *input.json* (Python → C++):

```
{
  "turn": 12,
  "difficulty": "HARD",
  "action": "DEPLOY",
  "parameters": {"unit_id": 2}
}
```

2) *state.json* (C++ → Python):

```
{
  "grid_size": [8, 8],
  "fires": [
    {"position": [2,3], "type": "NORMAL", "intensity": 50},
    {"position": [5,4], "type": "COPAS", "intensity": 90}
  ],
  "units": [
    {"id": 2, "type": "CAMION", "status": "READY"}
  ],
  "civilians": [
    {
      "id": 205,
      "type": "ANCIANO",
      "position": [4,5],
      "status": "ALIVE"
    }
  ],
  "risk_zones": [
    {"position": [2,3], "risk": 50},
    {"position": [5,4], "risk": 135}
  ],
  "game_stats": {
    "score": 1250,
    "turn": 12,
    "civilians_alive": 3,
    "civilians_dead": 1,
    "defeat": false
  }
}
```

H. User Interface Wireframe

The Pygame interface displays an 8×8 grid with forest (green), normal fire (red), canopy fire (dark red), and civilians (blue). The HUD shows turn, score, available units with deploy buttons, and action buttons for evacuation and ending the turn.



Fig. 4. User Interface Wireframe

III. RESULTS AND DISCUSSION

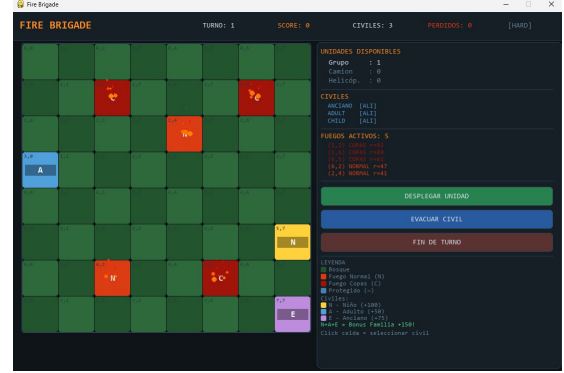


Fig. 5. 8×8 Game Grid with Active Fires (Normal and Canopy) and Civilians

The following experiments are planned to validate the system:

- **Unit Tests:** Verify Linked List (enqueue, dequeue, isEmpty) and BST (insert, findMax, remove, update) operations.
- **Greedy Algorithm Tests:** Confirm highest-risk cell selection with proper tie-breaking.
- **Backtracking Algorithm Tests:** Validate route finding in various fire configurations.
- **Integration Tests:** Ensure correct JSON communication between C++ and Python.
- **Gameplay Tests:** Evaluate difficulty balance and variant impact.

Upon completion, quantitative results including performance metrics and success rates will be presented.

IV. CONCLUSIONS

This paper has presented the complete design of Fire Brigade, a turn-based simulation game that implements Greedy and Backtracking algorithms within a modular C++/Python/Pygame architecture. The system includes a Linked List for unit dispatch, a BST for risk-based cell ordering, a Greedy dispatcher, a Backtracking evacuator, three difficulty levels, three gameplay variants, a complete scoring system, win/loss conditions, a defined turn structure, and a JSON communication protocol.

The work provides a complete, detailed, and reproducible design demonstrating how classical algorithms can be integrated into an interactive system. Future work includes completing the implementation, conducting tests, balancing difficulty levels, and adding features such as save/load functionality.

REFERENCES

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [2] Trulli, E., Pizzorni, D., & Parodi, G. (2007). *Forest fire simulation model using cellular automata*. Proceedings of the 2007 Summer Computer Simulation Conference, 1–6.